

the wondrous curse of interoperability

*what interop is,
and how to cope*



steve loughran
hp laboratories
slo@hplb.hpl.hp.com
january 2003

why is S/W dev like mountaineering?



all looks fine from 30,000'

vision



**up close
it's terrifying**

reality

and it is too late to turn back



commitment!

how do you survive?



- equipment
- process
- practice
- planning
- fear management

equipment



- good CPU, lots of RAM, HDD, WLAN...
- IntelliJ IDEA
- WSDL editor
- Ant+JUnit/HttpUnit/Cactus
- cellphone & caffeine
- *Continuous Integration*
- *Automated Deployment*

process, planning, preparation



- alpine starts
- speed
- maps, milestones, checkpoints
- practice: crevasse rescue, first aid, navigation in zero-visibility, snow-caves...
- *heaviest person goes first*

the Web Services vision



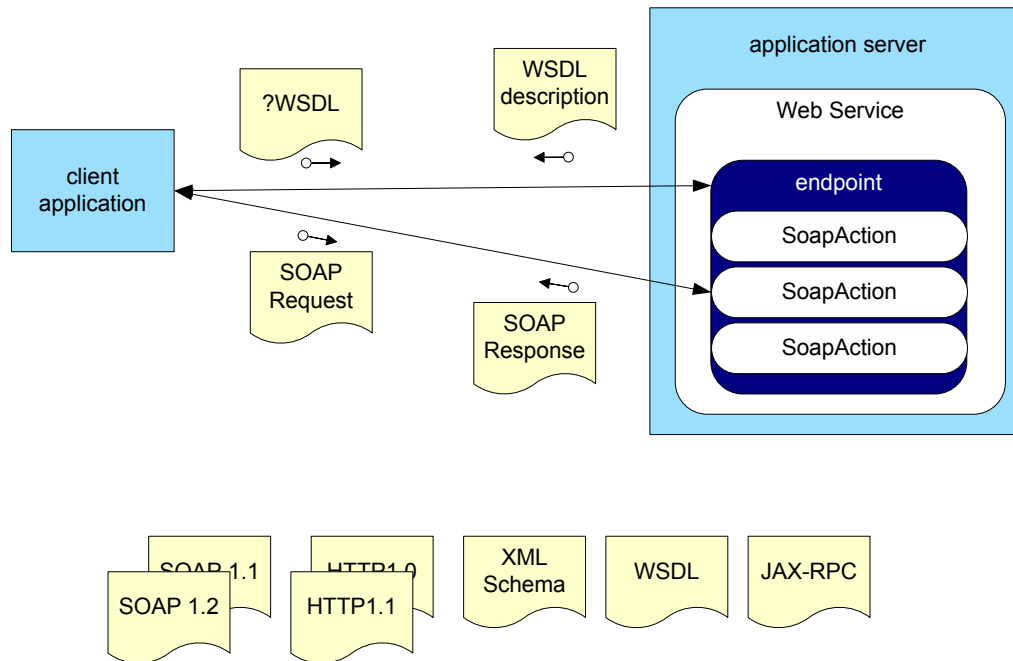
XML over HTTP lets
everything talk to everything
else



problem:

sometimes it gets a bit
"technical"

Web Services: quick summary



.NET
Axis
Pocket-SOAP
JAX-RPC RI
SOAP::Lite
kSOAP
gSOAP
MS STK

...

many specs

**many languages + many implementations =
interop problems**

key problem: naïve and unrealistic expectations

- you can't send things like `java.io.File` over the wire.
- you shouldn't send serializable objects like `javax.swing.JPanel` over the wire.
- you can't do transactions (yet)
- there is no good authentication system (yet? ever?)

***SOAP is not Corba, RMI, RMI-IIOP, .NET remoting, DCOM(+), or any
other classic
remote object invocation mechanism***

primitive data types

`uint, ulong, char`

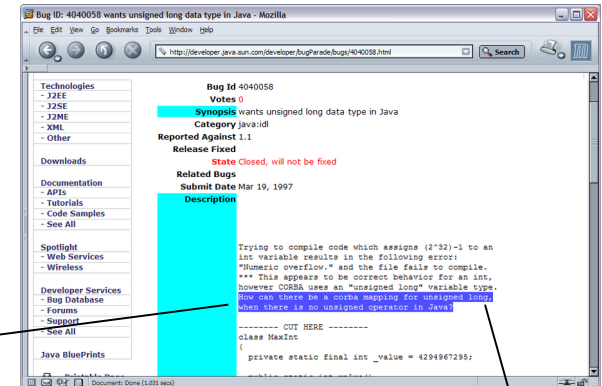
`[WebMethod]`

```
public ulong Increment(ulong counter) {  
    return counter+1;  
}
```

`[WebMethod]`

```
public char GetChar() {  
    return (char) (32+random.Next(64));  
}
```

Java bug database bug # 4186775



"The omission of unsigned integral types from Java is somewhat controversial, but they are in fact inessential, if perhaps convenient nonetheless."

enumerations

C# bitfield enums

```
[Flags]
public enum Rating : int {
    unknown=0, fun=1, scary=2, hard=4, painful=8
}

[WebMethod]
public Rating HowWasIt(String route) {
    return (Rating)random.Next((int)Rating.painful*2);
}
```

are marshalled as an XML Schema list

```
<>fun scary hard</>
```

that many things (Axis, Castor) cannot handle yet

exceptions

- how do you send exceptions down the wire
- how does the far end handle them?
- what about extra data in a SoapFault?
- what about languages with no notion of exceptions?

JAX-RPC tries to send any Exception over the wire.

axis: throw AxisFault with info in the details

framework objects and collections

- ArrayList
- DataSet
- java.util.HashMap

"the hashtable problem"

sessions

```
[WebMethod(  
    EnableSession=true  
)]
```

```
stub.  
    setMaintainSession(true)
```

1. cookies
2. SoapHeaders
3. dynamic endpoints
4. sessionID in API

- W3C push for SOAP GET is incompatible with SOAP Headers
- WSDL for dynamic endpoints?
- we need a SoapHeader standard

binaries:

- SOAP with Attachments (SwA) is in JAX-RPC
- DIME is in WS-toolkit for .NET
- Axis: SwA and DIME (but not WSDL)
- base-64 is the fallback

***no attachments in
Basic .NET 1.0***

	SwA	DIME
Axis	✓	✓
Sun JWSRP	✓	✗
.NET 1.0	✗	✗
.NET WSE	✗	✓

other surprises

- RPC/ENC versus DOC/LIT
- floating point: decimal places, infinity
- DateTime & timezones
- object references
- ragged arrays
- Sun's JAX-RPC reference implementation bails out if it can't handle *all* methods at an endpoint.

why did we get into this mess?

- Web Services grew by accretion, not planning.
- nobody wants to scare developers off.
- coding 'IDL' is considered harder than exporting methods.
- closed world systems (DCOM, RMI) don't have interop issues.

***there is no point writing a
Web Service unless you
want interoperability***

who can fix this?

the architects: W3C, WS-I

the engineers: SoapBuilders

you

stay out of trouble



1. understand the nature of the problem
2. avoid the big issues
3. drive off the XSD & WSDL
4. test for interop

avoid the big issues

- no unsigned datatypes, char, enums
- don't send fancy objects
- stick to your own simple 'structs' and arrays
- support multiple binary upload mechanisms
- worry about sessions

start with the XML Schema

- the schema defines your serialization format
- this should be independent of use (files, SOAP, REST)
- but you *must* use the subset of that interoperates over SOAP.

drive off the WSDL

- create a WSDL file using the Schema
- create server stubs from the WSDL
- just as if you were writing an IDL specification
- only uglier

test for interop

- pick the likely 'other' platforms
- generate client bindings from the WSDL
- write test cases to call the service.
- compile, run, fix
- repeat

ant

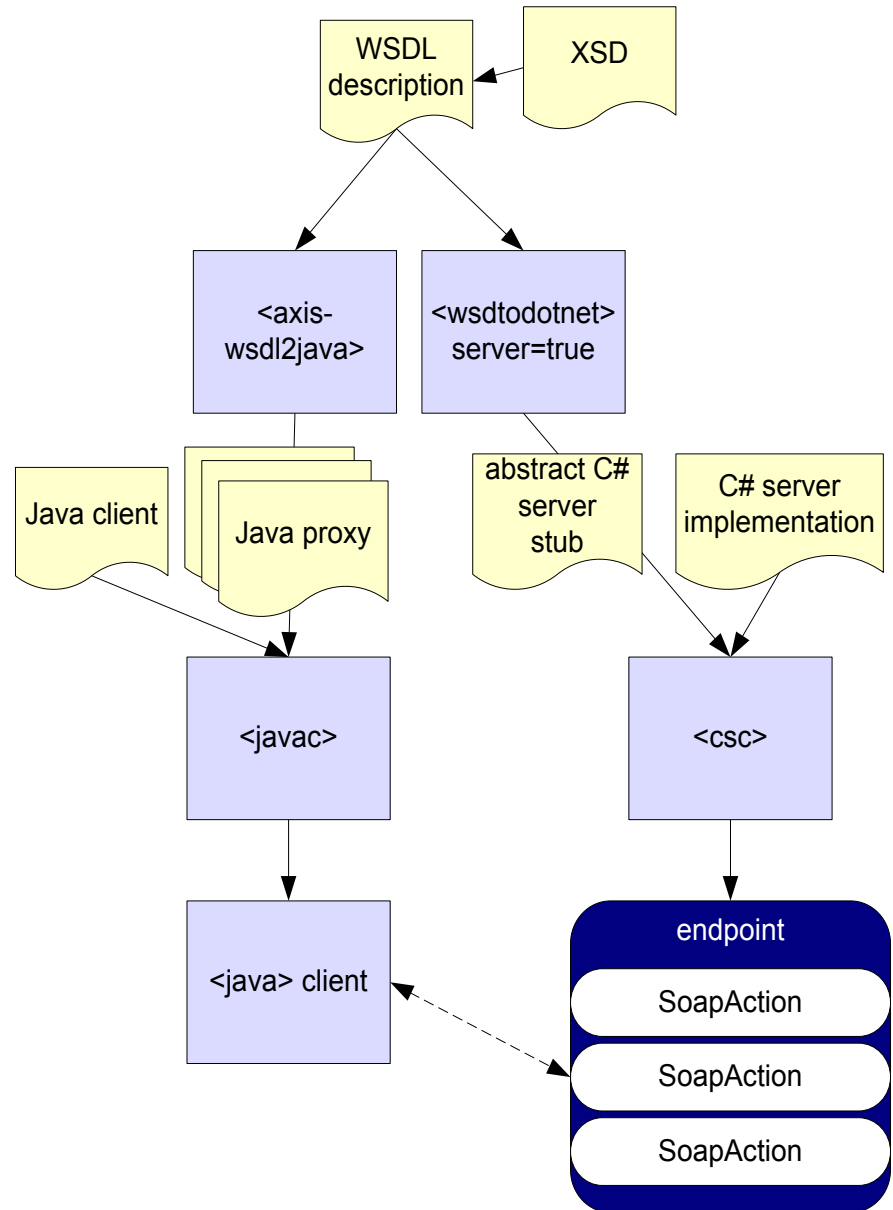
- jakarta.apache.org/Ant
- imports WSDL
 - `<wsdltodotnet>`
 - `<axis-wsdl2java>`

automated build and test with
CruiseControl, Gump, Anthill

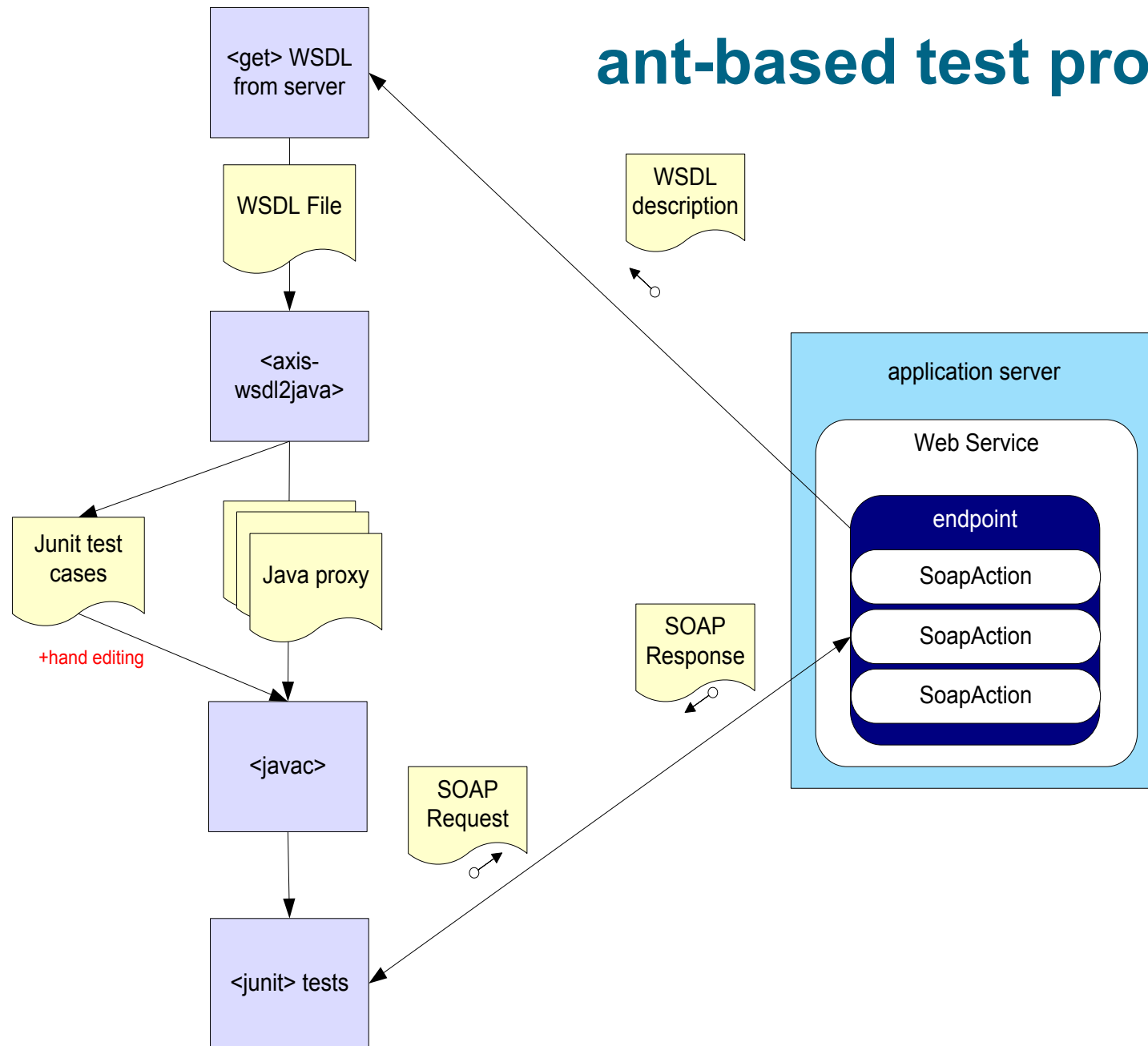
JUnit

- unit test framework from junit.org
- httpunit: adds http functional testing
- Axis can generate JUnit tests from WSDL

drive off the WSDL



ant-based test process



result: you know what to avoid

Unit Test Results: EndpointTestCase - Microsoft Internet Explorer

File Edit View Favorites Tools Help

Address D:\Projects\WebServices\interop\demo\client\build\test\reports\EndpointTestCase

Unit Test Results

Designed for use with [JUnit](#) and [Ant](#).

Class EndpointTestCase

Name	Tests	Errors	Failures	Time(s)
EndpointTestCase	14	0	5	7.109

Tests

Name	Status	Type	Time(s)
test1EndpointSoapVectorize	Success		4.109
test2EndpointSoapBoxify	Success		0.141
test3EndpointSoapIncrement	Success		0.187
test4EndpointSoapGetSessionCallCount	Failure	expected:<2> but was:<1> junit.framework.AssertionFailedError: expected:<2> but was:<1> at EndpointTestCase.test4EndpointSoapGetSessionCallCount (EndpointTestCase.java:98) at sun.reflect.NativeMethodAccessorImpl.invoke0 (Native Method) at sun.reflect.NativeMethodAccessorImpl.invoke (NativeMethodAccessorImpl.java:39) at sun.reflect.DelegatingMethodAccessorImpl.invoke (DelegatingMethodAccessorImpl.java:25)	0.156
test5EndpointSoapSetSessionMessage	Failure	expected:<1> but was:<null>	0.157

Done My Computer

summary

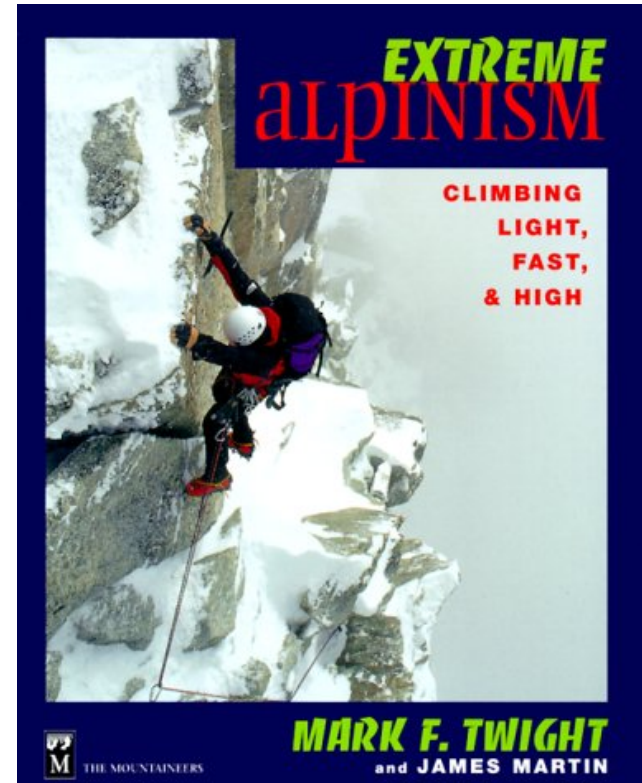
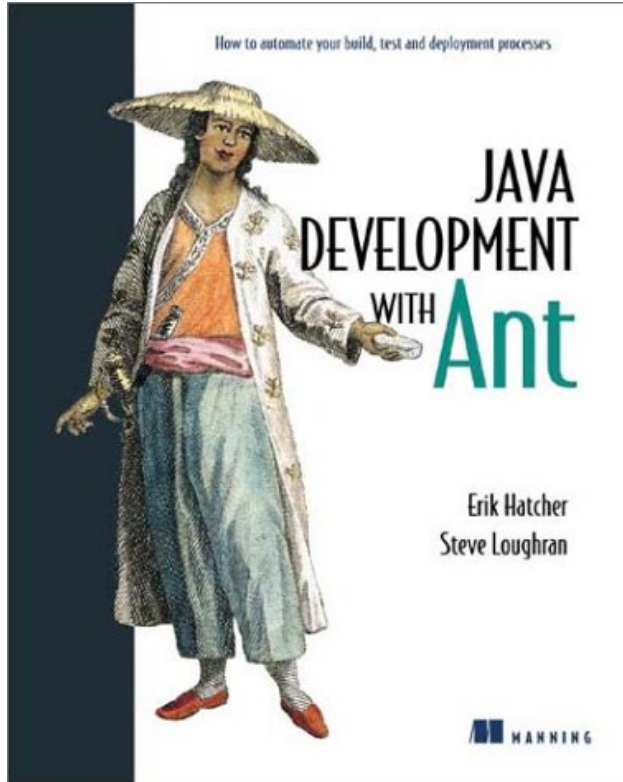
interop issues:

find them

before you fall into
them



read these



and maybe these

